

JAVA ASSIGNMENT



NAME : K.SIVA KAVYA

REGD.NO : 24501A05A3

BRANCH : CSE

SECTION : II

COLLEGE : PRASAD V POTLURI SIDDGARTHA INSTITUTE OF TECHNOLOGY

SELF INTRODUCTION

RESUME LINK :

https://drive.google.com/file/d/1YgqAORfTfvVQvTeX2nrqYzQW2aNglrR9/view?usp=drive_link



CASE STUDY TITLE

**SMART TRAFFIC
LIGHT CONTROLLER**



PROBLEM STATEMENT:

In modern urban areas, increasing traffic congestion has become a major concern, causing delays, fuel wastage, and higher pollution levels. Conventional traffic light systems operate on fixed time intervals without considering real-time traffic conditions, resulting in inefficient flow of vehicles and longer waiting times. Pedestrian crossings are often not synchronized effectively, leading to potential safety risks and further traffic delays.

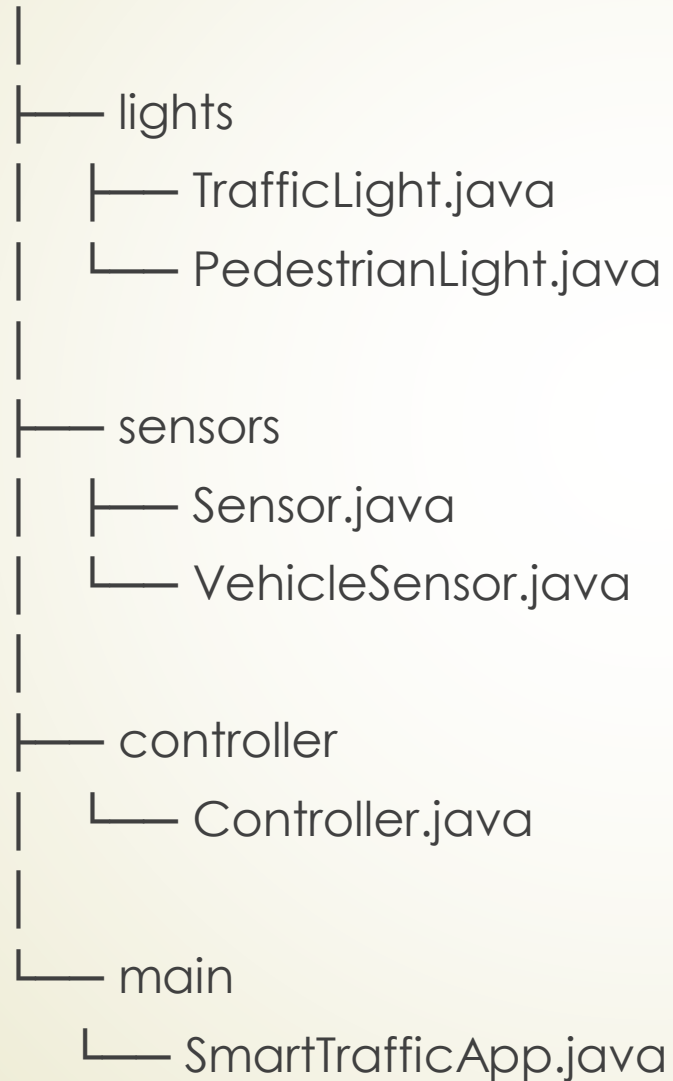
The main problem, therefore, is to develop a **Smart Traffic Light Controller** that automates and optimizes the switching of traffic lights based on live input and scheduling constraints. It should model the interaction between vehicles, pedestrians, and sensors effectively while maintaining safety, adaptability, and reliability. This problem involves integrating real-time data processing, concurrent operations, and robust error handling to simulate a realistic, efficient, and fault-tolerant traffic management environment suitable for modern smart cities.

PROBLEM SOLUTION:

- The solution involves designing a **Smart Traffic Light Controller System** that automates traffic management using real-time sensor data and intelligent scheduling. The system is built using object-oriented principles with key classes such as Traffic Light, Pedestrian Light, Sensor, and Controller, along with interfaces like Controllable and Schedulable to standardize control and scheduling operations.
- Each Traffic Light manages its states—RED, YELLOW, and GREEN—with timings adjusted dynamically based on input from Sensor objects that detect vehicle presence and queue length. The Pedestrian Light class extends Traffic Light, adding WALK and DON'T WALK signals to ensure pedestrian safety. The Controller class coordinates all lights, adjusting signal durations to optimize traffic flow and prevent congestion across multiple intersections.
- This solution creates an adaptive, efficient, and fault-tolerant traffic control system that improves traffic flow, reduces waiting times, and enhances pedestrian safety. By integrating real-time monitoring, multithreading, and intelligent scheduling, it provides a practical framework for modern smart city traffic management.

Package Structure

com .smarttraffic



UML Class

1. TrafficLight Class

```
classDiagram
    class TrafficLight {
        -color: String
        -location: String
        -duration: int
        +changeColor(): void
        +run(): void
    }
    TrafficLight <|-- PedestrianLight
    TrafficLight <|-- Sensor
    TrafficLight <|-- Controller
```

UML Class Diagram for TrafficLight Class:

- Attributes:
 - color: String
 - location: String
 - duration: int
- Operations:
 - + changeColor(): void
 - + run(): void
- Generalization:
 - «implements Controllable, Runnable»

2. PedestrianLight Class (extends TrafficLight)

```
classDiagram
    class PedestrianLight {
        -buttonPressed: boolean
        +requestCrossing(): void
        +changeColor(): void
    }
    PedestrianLight <|-- TrafficLight
```

UML Class Diagram for PedestrianLight Class (extends TrafficLight):

- Attributes:
 - buttonPressed: boolean
- Operations:
 - + requestCrossing(): void
 - + changeColor(): void
- Generalization:
 - «extends TrafficLight»

3. Sensor Class

```
classDiagram
    class Sensor {
        -id: String
        -isActive: boolean
        +detectVehicle(): boolean
        +detectFailure(): void
    }
    Sensor <|-- TrafficLight
    Sensor <|-- PedestrianLight
    Sensor <|-- Controller
```

UML Class Diagram for Sensor Class:

- Attributes:
 - id: String
 - isActive: boolean
- Operations:
 - + detectVehicle(): boolean
 - + detectFailure(): void
- Generalization:
 - «implements Controllable, Runnable»

4. Controller Class

```
classDiagram
    class Controller {
        -lights: List<TrafficLight>
        -sensors: List<Sensor>
        -trafficQueue: Queue<String>
        +scheduleLights(): void
        +handleSensorData(): void
        +startSystem(): void
    }
    Controller <|-- TrafficLight
    Controller <|-- PedestrianLight
    Controller <|-- Sensor
```

UML Class Diagram for Controller Class:

- Attributes:
 - lights: List<TrafficLight>
 - sensors: List<Sensor>
 - trafficQueue: Queue<String>
- Operations:
 - + scheduleLights(): void
 - + handleSensorData(): void
 - + startSystem(): void
- Generalization:
 - «implements Controllable, Runnable»

Controllable

```
classDiagram
    class Controllable {
        +changeColor(): void
    }
```

UML Class Diagram for Controllable Interface:

- Operations:
 - + changeColor(): void

Schedulable

```
classDiagram
    class Schedulable {
        +scheduleLights(): void
    }
```

UML Class Diagram for Schedulable Interface:

- Operations:
 - + scheduleLights(): void